



KTH Royal Institute of Technology

BUILDING NETWORKED SYSTEMS SECURITY EP2520

System Design and Implementation Details for ACME

March 2025

Contents

1	Networked System Design	1
1.1	Basic	1
1.2	Employee Authentication	1
1.3	Secure Connectivity	1
1.4	Confidentiality and Anonymity	1
1.5	Secure Wireless Access	2
1.6	Secure File Exchange	2
1.7	Other Security	2
2	Networked System Implementation	2
2.1	Topology and Explanation	2
2.2	Basic	3
2.3	Employee Authentication	3
2.4	Secure Connectivity	3
2.5	Confidentiality and Anonymity	4
2.6	Secure Wireless Access	4
2.7	Secure File Exchange	4
2.8	Other Security	4
3	A simulating attack	5
3.1	Simulating an Attack (Red Team Perspective)	5
3.2	Attack Detection (Blue Team Perspective)	5

1 Networked System Design

1.1 Basic

Our network infrastructure is built using the provided routers, switches, and cables to establish the fundamental network connections. To enhance network functionality and security, we configure the routers with **OpenWrt**. Based on OpenWrt, we can have DNS services, ensuring efficient domain name resolution and secure filtering of domain requests. The routing functionality is also included to facilitate communication between different network segments while enforcing traffic policies. Additionally, DHCP services can be used to dynamically allocate IP addresses, ensuring efficient device management and network scalability. The built-in firewall of OpenWrt can also be utilized in the following parts.

1.2 Employee Authentication

Our employee authentication system is designed using **FreeIPA** and **RADIUS** to ensure secure and efficient identity management. FreeIPA provides centralized authentication, authorization, and account management by integrating LDAP, Kerberos, and other security mechanisms to enforce robust access control policies. Meanwhile, RADIUS (Remote Authentication Dial-In User Service) acts as an authentication proxy, verifying user credentials against FreeIPA before granting network access.

1.3 Secure Connectivity

Our secure connectivity solution is built using **OpenVPN** to establish a reliable and encrypted communication channel between different locations. Employees visiting the London office should be able to securely connect to the Web Server in Stockholm through an OpenVPN tunnel, ensuring data confidentiality and integrity. Access to our internal infrastructure is restricted to ACME employees only, and only computers with IP addresses assigned to the Stockholm headquarters or the London branch are allowed to connect. Remote access is carefully managed, using strong authentication mechanisms and encrypted tunnels to prevent unauthorized access. To facilitate secure employee-to-employee communication, we integrate **Rocket.Chat** into our infrastructure, ensuring that all internal discussions remain encrypted and protected within the private network.

1.4 Confidentiality and Anonymity

In this part, we also plan to use **OpenVPN** to ensure secure and encrypted communication between the branch and the headquarters, preventing unauthorized third-party access. The main web server, which contains critical corporate data, is only accessible to trusted users, such as employees at the London branch and Stockholm headquarters. To enforce different levels of access control across web servers, we plan to use firewall in OpenWRT router, which allows us to define strict firewall rules ensuring that only authorized users can reach specific servers based on their security classification. Less critical web servers can be accessed by employees under specific conditions, such as remote access from their personal laptops.

1.5 Secure Wireless Access

Our secure wireless access solution is planned to be built upon **RADIUS** to ensure robust authentication and authorization for employees connecting via Wi-Fi. Visiting employees in London should be able to securely connect to the Stockholm network using their laptops and a wireless connection. The RADIUS server enforces authentication by verifying user credentials against our central identity management system, ensuring that only authorized personnel gain access.

1.6 Secure File Exchange

Our secure file exchange system is planned to use **ownCloud** to ensure the confidentiality, integrity, and authenticity of file transfers between employees. Authentication is enforced through ownCloud's built-in access control mechanisms, ensuring that only authorized personnel can log in and use the platform.

1.7 Other Security

When talking about other security, **Snort** combined with **Splunk** are supposed for intrusion detection and monitoring, enabling real-time analysis of network traffic and detecting potential threats through signature-based and anomaly-based detection methods. **Hfish** will be utilized for honeypot deployment, simulating vulnerable systems to attract and analyze malicious activities, allowing us to gain intelligence on attack patterns and mitigate potential threats before they reach critical infrastructure. Additionally, the built-in firewall of **OpenWrt** is configured to provide DDoS attack mitigation, implementing traffic filtering, rate limiting, and anomaly detection to prevent large-scale denial-of-service attacks.

2 Networked System Implementation

2.1 Topology and Explanation

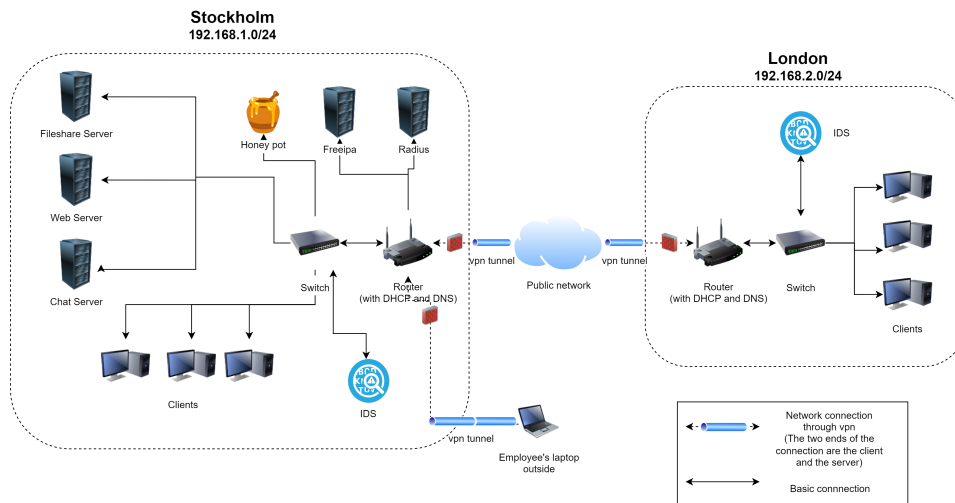


Figure 1: Topology of Network

We have two networks one in Stockholm and one in London. Stockholm is on 192.168.1.0/24 and London is on 192.168.2.0/24, both are connected to each other via OpenVPN tunnel. Any traffic to internal services can go from one network to another via the tunnel.

Both the networks have their own DNS, DHCP and IDS systems and both connect to the external web via their own routers. This is done to split the load between the two routers.

Employees can connect to the network from outside office as well using OpenVPN Connect application. This will form a tunnel between the Stockholm network and the employees device enabling the employees to access internal services in the stockholm network.

2.2 Basic

For the fundamental setup of our network infrastructure, we have flashed OpenWrt onto two routers and performed essential configurations to enable core networking functionalities. OpenWrt, an open-source Linux-based firmware, allows for customizable and secure network management.

After successfully installing OpenWrt, we configured the routers to support DNS, DHCP, and firewall services, ensuring a stable and functional network environment. The DNS service resolves domain names efficiently, while the DHCP server dynamically assigns IP addresses to connected devices. The firewall is configured to enforce security policies, blocking unauthorized traffic and protecting internal network resources. To facilitate remote management, we also enabled SSH access to the routers. This allows us to securely connect to the OpenWrt system via the terminal, where we can modify configurations, troubleshoot issues, and apply updates as needed.

2.3 Employee Authentication

The architecture uses a freeIPA server as its central point of authentication and identity management. The server also acts as the central Certificate Authority. We configured identities for employees, each having a unique id, a password, and a certificate issued by the CA. We also configured groups for each service in the architecture that require authentication, i.e. for freeRadius, ownCloud, and web servers. A freeRadius server was set up to act as an authentication proxy for freeIPA. It handles authentication to company Wi-Fi. The freeIPA server is also configured to use One-Time Passwords for authentication with username and password. Employees who connect to a service without a certificate can thus log in with their credentials and an OTP that they can access through Google authenticator or some similar app on their phone. Otherwise, users authenticate themselves with certificates through eap-tls.

2.4 Secure Connectivity

Secure connectivity is achieved between the two networks by using public key cryptography. With the help of a CA which is established in the Stockholm network both Stockholm and London have their private public key pair and a certificate from the CA. These are used to authenticate each other and avoid any man in the middle attack while there is a TLS handshake between the two to establish a OpenVPN tunnel.

We also have deployed Rocket.Chat to ensure secure communication among employees. The deployment was done using Snaps for a simplified installation process. After installing Rocket.Chat on a Linux server, we configured user authentication based in it to enhance security.

2.5 Confidentiality and Anonymity

We achieve confidentiality and anonymity on the OpenVPN tunnel between the London and Stockholm network.

Confidentiality is achieved because the tunnel is encrypted using AES standard so no one on the open internet can decrypt and read the contents of the packets that are being forwarded.

Anonymity is achieved because to anyone observing the packets would only find source and destination as Stockholm and London routers but would not be able to find who is sending and receiving the packet behind the networks.

2.6 Secure Wireless Access

To ensure secure wireless access, we have implemented RADIUS-based authentication using EAP-TLS (Extensible Authentication Protocol - Transport Layer Security). This approach enables strong certificate-based authentication, eliminating the need for passwords and providing a higher level of security for user access.

2.7 Secure File Exchange

For the implementation of secure file exchange, we have deployed an ownCloud service on a virtual machine, integrating it with MongoDB for efficient data management. The deployment process involves setting up a virtual machine running a Linux-based operating system, installing the required dependencies, and then configuring ownCloud to manage file storage and user access. The system ensures security by leveraging ownCloud's built-in access control mechanisms, where users are authenticated and assigned specific roles with predefined permissions to regulate file sharing activities. The integration with MongoDB provides a scalable and high-performance backend for storing file metadata and user information. Furthermore, access policies are configured within ownCloud to restrict unauthorized file exchanges.

2.8 Other Security

We deployed Snort 3 as our primary IDS to monitor network traffic and detect potential threats. Snort is configured to analyze network packets and identify malicious activities based on predefined rules. To enhance threat visibility and response, we integrated Snort with Splunk, a Security Information and Event Management (SIEM) system. Snort logs are formatted in a way that is compatible with Splunk, enabling data ingestion. In Splunk, a custom dashboard has been created to visualize IDS alerts, allowing the security team to monitor network security events in real-time and respond to incidents efficiently.

To further strengthen our security monitoring, we deployed HFish, a high-interaction honeypot designed to detect unauthorized access attempts. HFish provides valuable insights into attack techniques and threat actor behaviors. The deployment process

includes installing HFish on a dedicated server, configuring various honeypot services (such as SSH, MySQL, and RDP traps), and enabling logging mechanisms to capture attack data.

To protect against Distributed Denial of Service (DDoS) attacks, we have configured our firewall with multiple security mechanisms. Traffic filtering rules have been implemented to block unnecessary services and restrict access to critical systems.

3 A simulating attack

3.1 Simulating an Attack (Red Team Perspective)

To verify the effectiveness of our IDS and honeypot system, we simulated an attack from a compromised internal machine:

Scan the internal network

The attacker performs a reconnaissance scan across the intranet to identify active hosts and open services.

Identify an easy target

The attacker finds an exposed system, which is actually a honeypot decoy (HFish).

Perform a brute-force SSH attack

Using `hydra`, the attacker attempts to brute-force SSH login credentials on the decoy machine.

Directory brute-force scanning

The attacker uses `gobuster` to scan for hidden directories and services running on the web decoy.

Trapped in the honeypot

The attacker unknowingly interacts with the honeypot, triggering alerts and logging activities.

3.2 Attack Detection (Blue Team Perspective)

Here's how Security Operations Center (SOC) of the ACME detect the attack:

Monitor the network via Splunk

The security team (blue team) observes Snort alerts ingested into the Splunk dashboard.

Detect the attack

The suspicious activities and brute-force attempts are identified in near real-time on Splunk.

Correlation with honeypot logs

Logs from HFish confirm that the attacker interacted with honeypot services (e.g., SSH decoy and fake web services).

Incident response

The security team can now block the attacking IP and perform further forensics.

Technical Details

OpenVPN

Here are the OpenVPN configurations that we have used

client.ovpn

```
1 client
2 dev tun
3 proto udp
4 remote <remote address> <remote port>
5 resolv-retry infinite
6 nobind
7 persist-key
8 persist-tun
9 cipher AES-256-CBC
10 auth SHA256
11 #comp-lzo
12 auth-user-pass
13 verb 3
14
15 <ca>
16 -----BEGIN CERTIFICATE-----
17 xxxxxxxxxxxxxxxx
18 -----END CERTIFICATE-----
19 </ca>
20
21
22 \subsection*{Snort3}
23 % \subsubsection*{System Preparation}
24 % Before installing Snort 3, update the system and install required \
    dependencies:
25
26 % \begin{lstlisting}[style=BashStyle]
27 % sudo apt-get update && sudo apt-get dist-upgrade -y
28 % sudo apt-get install -y build-essential autotools-dev libdumbnet-dev \
    \
29 %     libluajit-5.1-dev libpcap-dev zlib1g-dev pkg-config libhwloc-dev \
    \
30 %     cmake liblzma-dev openssl libssl-dev cpputest libsqlite3-dev \
31 %     libtool uuid-dev git autoconf bison flex libcmocka-dev \
32 %     libnetfilter-queue-dev libunwind-dev libmnl-dev ethtool
33 %
```

Installing Snort 3

```
1 cd ~/snort_src
2 wget https://github.com/snort3/snort3/archive/refs/tags/3.1.5.0.tar.gz
3 tar -xzf 3.1.5.0.tar.gz
4 cd snort3-3.1.5.0
5 ./configure_cmake.sh --prefix=/usr/local --enable-tcmalloc
6 cd build
7 make
8 sudo make install
```

FreeIPA

Installation (Fedora Linux)

```
1 $ hostnamectl status
2 $ hostnamectl set-hostname ipa.example.com
3 $ vim /etc/hosts
4
5 127.0.0.1      <ipa server address> ipa
6 $ firewall-cmd --add-service=freeipa-ldap --add-service=freeipa-ldaps
7 $ firewall-cmd --add-service=freeipa-ldap --add-service=freeipa-ldaps \
  --permanent
8 $ firewall-cmd --reload
9 $ dnf install -y freeipa-server
10 $ dnf install -y freeipa-server-dns
11 $ ipa-server-install
12 $ kinit admin
13 now visit https://ipa.example.com
```

Then create a bind user for freeRADIUS and other services to connect to.

```
1 $ echo "dn: uid=bind-user,dc=example,dc=com
2 changetype: add
3 objectclass: account
4 objectclass: simplesecurityobject
5 uid: bind-user
6 userPassword: <password>" > bind-user.ldif
7
8 $ ldapmodify -H <ipa server address> -x -D "cn=Directory Manager" -w \
  ['<password>'] -f bind-user.ldif
```

FreeRADIUS

Installation (Fedora Linux)

```
1 $ sudo dnf install freeradius freeradius-utils freeradius-ldap \
  freeradius-krb5
```

Clients

Each client that wants to use the server has to be added to clients.conf.

```
1 $ vi /etc/raddb/clients.conf
2 client stockholm {
3     ipaddr = <client-ip>
4     proto = *
5     secret = <secret>
6     nas_type = other
7     limit {
8         max_connections = 16
9         lifetime = 0
10        idle_timeout = 30
11    }
12 }
```

LDAP

LDAP needs to be enabled to query the freeIPA server. Both `/etc/raddb/sites-enabled/default` and `/etc/raddb/sites-enabled/inner-tunnel` need to be updated with the following configuration:

```
1 .
2 .
3 #-ldap
4 ldap
5 if ((ok || updated) && User-Password) {
6     update {
7         control:Auth-Type := ldap
8     }
9 }
10 .
11 .
12 Auth-Type LDAP {
13     ldap
14 }
15 .
16 .
```

And the LDAP module needs to be enabled.

```
1 $ ln -s /etc/raddb/mods-available/ldap /etc/raddb/mods-enabled/
```

Then, edit `/etc/raddb/mods-enabled/ldap` with the following configuration.

```
1 .
2 .
3 ldap {
4     server = '<ipa server address>'
5     identity = 'uid=<bind-user>,dc=example,dc=com'
6     password = '<password>'
7     base_dn = 'dc=example,dc=org'
8     user {
9         base_dn = "cn=users,cn=accounts,${..base_dn}"
10        filter = "(uid=%{%{Stripped-User-Name}:-%{User-Name}})"
11    }
12    group {
13        base_dn = \
14            "cn=<radius_group>,cn=groups,cn=accounts,${..base_dn}"
15        filter = "(objectClass=groupOfNames)"
16    }
17    .
18    .
```

Enable EAP-TLS

First, enable EAP.

```
1 ln -s /etc/raddb/mods-available/eap /etc/raddb/mods-enabled/eap
```

Then import the CA certificate and generate server key and certificate to `/etc/raddb/certs`. Lastly edit `.` to point to the correct certificate and key files.

```
1 .
2 .
```

```
3  tls-config tls-common {  
4      private_key_file = /etc/raddb/certs/server.key  
5      certificate_file = /etc/raddb/certs/server.crt  
6      ca_file = /etc/raddb/certs/ca.pem  
7  }  
8  .  
9  .
```

README

User Authentication

This guide provides step-by-step instructions on how employees can securely connect to the company network using FreeIPA authentication.

1 Authentication Overview

Our authentication system is managed by a FreeIPA server, which acts as the central point for identity management and certificate authority (CA). Employees can authenticate using:

- Certificate-based authentication (EAP-TLS)
- Username & Password + One-Time Password (OTP)

2 Connecting to the Company Wi-Fi

If you **have** a certificate:

1. Ensure your device has the issued certificate and private key installed.
2. Connect to the company Wi-Fi and select EAP-TLS authentication.
3. Choose your certificate when prompted.

If you **do not have** a certificate:

1. Select the company Wi-Fi and choose EAP-PEAP authentication.
2. Enter your FreeIPA username and password.
3. Open Google Authenticator (or similar app) to generate a One-Time Password (OTP).
4. Enter the OTP when prompted.

3 Connecting to OpenVPN:

1. Download and install OpenVPN on your device.
2. Obtain your OpenVPN configuration file (.ovpn) from IT support.
3. If you have a certificate, place it in the appropriate directory (/etc/openvpn/client/ on Linux or in the OpenVPN app on mobile).
4. Open the OpenVPN client and import the configuration file.
5. Connect using one of the following methods:
 - Certificate Authentication: No password is required if using EAP-TLS.
 - Username & Password + OTP: Enter your FreeIPA credentials and OTP when prompted.

4 Accessing Web Services (ownCloud, Internal Web Servers):

1. Open a web browser and go to the service's URL (e.g., <https://acme.owncloud.com>).
2. If using a certificate, your browser will prompt you to select it for authentication.
3. If not using a certificate:
 - Enter your FreeIPA username and password.
 - Open Google Authenticator (or similar app) and enter the OTP when prompted.

Snort with Splunk

This guide explains how to use Snort 3 with Splunk to achieve real-time intrusion detection visualization.

We've already modified the Snort configuration file (snort.lua) to enable JSON logging. So you can create a Splunk Dashboard for Snort Alerts

1. Log in to Splunk Web at http://splunk_website:8000/.
2. In Splunk Search, use the following query to retrieve Snort alerts:

```
1 sourcetype="snort3:alert:json"
2 | table _time src_ap dst_ap msg
```

2. Click Search as to Dashboard Panel to visualize the results.
3. Customize the dashboard with charts, tables, or alerts for better insights.

Hfish User Guide

This guide explains how to use Hfish to deploy honeypot and monitor the attack to honeypots.

1. Web Dashboard Login

Open a browser and visit: http://jhfish_website:4433/web

Default credentials: Username: admin Password: HFish2021 HFish simulates the following services by default:

2. Honeypot Management

Here's some default services honeypot provides.

Service	Default Port
SSH	22
RDP	3389
MySQL	3306
Redis	6379
HTTP/HTTPS	80 / 443

3. Enable/Disable Services

1. Log into the web dashboard.
2. Navigate to **Honeypot Services** → **Service Management**.
3. Enable/disable or change ports for each service.
4. Click **Save & Restart**.

4. Web Dashboard

Go to **Event Center** → **Attack Events** to view:

1. Attacker IPs
2. Target services
3. Payloads and request types